FINDING THE OPTIMAL ALLOCATION TO MAXIMIZE THE BETA OF A PORTFOLIO TO RESPECT VALUE AT RISK ARBITRARY CONSTRAINT

Gianni Mazaud
Finance Student (Panthéon-Sorbonne University)
gianni.mazaud@icloud.com

June, 2026

ABSTRACT

Our base intuition is to maximize the aggressiveness of a given portfolio, represented by the β while minimizing tail risks, that is to say the Value at Risk. The idea is to benefit as much as possible from market momentum and be able to survive a temporary reversal without abandoning the position. We could somehow say that we want an asymmetric β that helps to grow during uptrend, but that does not imply high losses during crisis period. Assuming that, on the long-term, markets are increasing, this strategy aims at benefiting from that long-term uptrend and not be taken out during crisis period. Actually, the objective is to determine the optimal allocation that satisfies this strategy.

We assume that asset picking is already done successfully. We scrap five years of daily market data of each of the chosen assets, we define the given parameters and constraints. In our example, the assets are: Tesla, Airbus, Google, Apple, Air France, Ferrari, BMW, LVMH, Société Générale, Goldman Sachs, JPMorgan, Microsoft, General Dynamics, and Boeing. We let the portfolio value be \$500 and we allow for a maximal loss of \$50 under a confidence level α of 99%. We compute risk measures through a KDE (with a Gaussian kernel) with Monte Carlo method. Then we use an SLSQP algorithm that solves our objective under the constraints given.

Empirically, we find that freezing the stochastic simulation space allows the algorithm to successfully converge, yielding an aggressively optimized portfolio ($\beta \approx 1.31$). The model strictly respects the non-parametric tail-risk boundary, achieving a VaR(99%) of \$21.66, safely below the maximum acceptable loss limit of \$37.50. Finally, out-of-sample backtesting validates the strategy's risk-adjusted superiority, generating a Treynor ratio of 63.27% that significantly outperforms the benchmark's 43.63%, proving the viability of capturing asymmetric market upside while surviving extreme tail-risk events.

Keywords Portfolio optimization · Kernel Density Estimator · Beta optimization · Value at Risk

1 Introduction

First of all, we shall define clearly the steps of our global optimization strategy. We should mention that our work is not about the optimization process itself but rather about the optimization objective and constraints.

The hypothesis we are formulating is that on the long-term, markets are rising. However, investing on the long-term opens the risk of being exposed to crisis. As we have seen throughout the years, and even in recent history, financial markets are exposed to major crisis. Hence, the objective is to take advantage of global markets growth and at the same time being able to survive major crisis.
In finance, we usually define risk as volatility. But it doesn't give any information about black swan events. Value at Risk and Expected Shortfall give a real idea of these events. This is why the constraints will be mostly about minimizing these risk measures as the constructed-portfolio primary objective is to survive crisis. Moreover, this portfolio is also constructed to be aggressive (i.e. $\beta > 1$) so that it profits from growth even more that the benchmark.

Finally, our major contribution is to find the right balance between taking advantage of markets' uptrend, and minimizing black swan events' risk.

Before going further, let's give some definitions. Value at Risk can be defined as: "a measure of the worst expected loss on a portfolio of instruments resulting from market movements over a given time horizon and a pre-defined confidence level". Formally, it can be written as:

$$VaR_\alpha = \inf\{l \in \mathbb{R} : \mathbb{P}(L \geq l) \geq l - \alpha\}$$

Expected Shortfall (also called Conditional Value at Risk—CVaR—) is defined as: ""a measure of the average of all potential losses exceeding the VaR at a given confidence level". Mathematically, we can write it as:

$$CVaR_\alpha = \mathbb{E}[L|L \geq VaR_\alpha]$$

As a reminder, under Basel III regulation, the confidence level for VaR is α = 99%. Through the FRTB, CVaR is observed at 97.5% level. This shift from VaR(99%) to CVaR(97.5%) is a way to strengthen the capital requirements as empirically we always find that CVaR(97.5%) is greater than VaR(99%).

To ensure a minimum diversification, we impose two allocation constraints: a minimum weight for each asset, and a maximum weight as well. The main constraint is about the maximum acceptable Value at Risk (MAVaR).

2 Kernel Density Estimation (KDE) in Return Distributions

Using a previous work we established, to measure these risk measures we will use a Kernel Density Estimator (KDE) with Monte Carlo method. The formal mathematical formula for a KDE is:

$$\hat{f}(x) = \frac{1}{nh} \sum_{i=1}^{n} K\left(\frac{x - x_i}{h}\right) \tag{1}$$

Where h is the bandwidth, $K(.)$ the Kernel Function. From our previous study, we find that empirically, a Gaussian kernel is pretty accurate for our purposes. Gaussian kernel is written as follow:

$$K(u) = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}u^2} \tag{2}$$

Hence we have:

$$\hat{f}(x) = \frac{1}{nh\sqrt{2\pi}} \sum_{i=1}^{n} e^{-\frac{1}{2}\left(\frac{x-x_i}{h}\right)^2} \tag{3}$$

To decide the value of the bandwidth, we use the Silverman's Rule of Thumb that imposes:

$$h = \left(\frac{4\hat{\sigma}^5}{3n}\right)^{\frac{1}{5}}$$

The benefit of this method is that it is non-parametric and then it doesn't impose any arbitrary choice on parameters, it fits itself around the distribution curve.

With an empirical backtesting [1], we observe that among the tested approaches KDE is the one with the lowest violation rate. It has higher values for each measures and we can debate wether it is too conservative or not, but we assume here that is it sufficient for our study. The main importance is to avoid the underestimation of tail risks that a classic gaussian approach implies.

3 Market Exposure and Empirical Beta Calibration

3.1 Benchmark selection and data acquisition

As US markets are the most considered ones and thaÙt S&P500, encompassing most of the biggest companies, provides a broad, highly liquid and widely accepted baseline for capturing systematic risk (market exposure) across our chosen equities. For that reasons, it will be used as our benchmark. The data are acquired on an observation window from June 2021 to June 2026, providing five years of daily observations (roughly 1,250 observations). Data acquisition is done through Python with the "yfinance" library. Values retained are the closing prices.

3.2 Covariance-based beta estimation

β quantifies an individual asset's sensitivity to non-diversifiable market movements. To make it simple, a portfolio with $\beta = 1$, will move exactly the same as the market. A portfolio with $\beta = 2$ will have the same variations as the benchmark but two times higher. If $\beta \geq 1$, we say that the portfolio is aggressive, whereas if $\beta \leq 1$, we say that the portfolio is defensive.

Mathematically, it is calculated as the ratio of the covariance between the asset's return and the benchmark's return to the variance of the benchmark's return.

$$\beta_i = \frac{Cov(R_i, R_m)}{Var(R_m)} \tag{4}$$

This measure is done over realized (historical) data. In our study, β_i are estimated over the last five years. The formula is relatively easy to find (See Appendix X.) and is obtained through the Ordinary Least Squares econometric method. By computing a single and static beta across the multi-year window, the model assumes that the covariance structure and the linear relationship between the equities and the market are relatively time-invariant. These empirical β_i values are extracted to serve as the constant coefficients for the linear objective function in your optimization engine.

4 Optimization framework

4.1 Objective function

As we said, the objective of our work is to maximize the exposure to market risk, and so to the systematic risk. Hence, we define the following function:

$$g(\omega) = \sum_{i=1}^{n} \omega_i \beta_i \tag{5}$$

And the objective is to maximize this g function. However, the algorithm we are going to use in our Python program, the SLSQP optimization architecture, is a minimizing algorithm. Then, by defining:

$$f(\omega) = -\omega^T \beta$$

We have that minimizing $f(\omega)$ is the same as maximizing $g(\omega)$. This function is strictly linear, resulting in a constant gradient:

$$\nabla f(\omega) = -\beta$$

This isolates the computational intensity to the non-linear risk constraint space, ensuring stable solver convergence. The optimizer searches for continuous allocation weights within the possible allocations, $w \in]0, 1[^n$.

4.2 Linear and boundary constraints

We let our portfolio is unleveraged, fully invested, and long-only. It guarantees that 100% of the capital is invested. Mathematically:

$$\sum_{i=1}^{n} w_i = 1 \tag{6}$$

Assuming that stock picking is well-done, all stocks' weight are strictly positive. However, to ensure a certain amount of diversification we can set a minimum weight as well as a maximum weight. We can write:

$$w_{min} \leq w_i \leq w_{max}, \forall i = 1, ..., n \tag{7}$$

Because the objective function is strictly linear, maximizing beta without bounds would result in a trivial solution, the optimizer would simply allocate 100% of the capital to the single highest-beta asset.

5 Non-linear risk constraint (methodological contribution)

5.1 Stochastic convergence problem

Although the focus of this paper isn't about the optimization algorithm, let's briefly explain the Sequential Least Squares Programming (SLSQP). This algorithm navigates the optimization space by computing local gradients. It uses finite

difference approximations to measure how a tiny perturbation in the weight vector ($w + \epsilon$) impacts the constraint boundary.

If the constraint function draws random variables on every iteration, the constraint surface becomes stochastic. Evaluating the exact same weight vector twice will yield slightly different VaR outputs due to simulation noise. When the simulation variance exceeds the marginal impact of the weight perturbation (ϵ), the gradient calculation collapses. The algorithm interprets the random noise as structural curvature, calculating erratic and false derivatives.

That this stochastic instability prevents the optimizer from executing smooth line searches or satisfying the Karush-Kuhn-Tucker (KKT) optimality conditions, inevitably leading to infinite loops, premature termination, or mathematically invalid corner solutions.

5.2 Freezing the state space (deterministic simulation)

To work correctly, the optimization algorithm needs to calculate local gradients without any random errors. Because of this, the Value at Risk (VaR) constraint must be completely deterministic (fixed and stable).If we input a specific set of portfolio weights, represented by w, the Kernel Density Estimation (KDE) and the Monte Carlo simulation must return the exact same risk number every single time. By keeping the simulation fixed, we remove the random "noise." This is a necessary step because random simulation noise confuses the algorithm, disrupts the gradient calculations, and stops it from finding the optimal allocation.

The main technical solution is to prepare the data in advance. Instead of creating new random numbers from the KDE while the algorithm is running, we must create a fixed table of simulated returns before starting the optimizer. This fixed table is an $N \times M$ matrix, where N is the number of Monte Carlo scenarios and M is the number of assets.

As a result, the mathematical surface of our problem becomes perfectly smooth because the random noise is gone. The SLSQP algorithm no longer gets confused by false variations. It can now correctly measure how a very small change in the portfolio weights $(w + \epsilon)$ affects the risk limit. This stability allows the optimization to work properly and successfully satisfy the necessary mathematical rules (the KKT conditions).

5.3 Tail Risk extraction

First, we find the total return of the portfolio for each possible situation. To do this, we simply multiply the chosen asset weights (w) by the simulated returns from our fixed table. Next, we organize all of these simulated portfolio returns in order, arranging them from the worst financial loss to the biggest gain.

Because we are using a 99% confidence level ($\alpha = 99\%$), we isolate the worst 1% of our sorted results. The specific threshold value that separates this worst 1% from the rest of the distribution is our exact Value at Risk.

Finally, this extracted VaR number is returned to the optimization algorithm. The algorithm checks if this risk is smaller or equal to our maximum allowed loss. If the risk is too high, the algorithm knows it must adjust the weights and try again.

6 Empirical results and performance analysis

6.1 Optimal allocation profile

The chosen stocks are: Tesla, Airbus, Google, Apple, Air France, Ferrari, BMW, LVMH, Société Générale, Goldman Sachs, JPMorgan, Microsoft, General Dynamics, and Boeing. Choices are made arbitrary, we take important companies that have shown performance over the long-term.

We decide to floor the minimum weight at 1.5% of the portfolio value, and on the opposite side to cap the maximum weight at 20% of it. (Having 14 shares given, an equally-weighted portfolio would give roughly 7% to each asset).

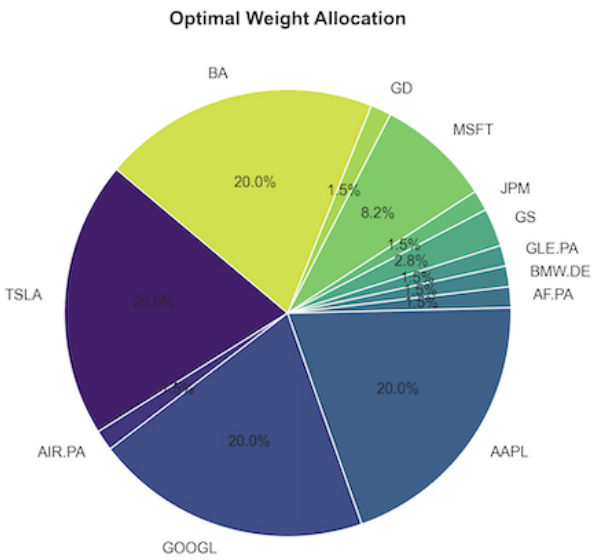

Figure 1: Optimal Weight Allocation

We see that our portfolio is mostly shared between four stocks, Apple, Google, Tesla, and Boeing. Microsoft also represents a small share of the optimized portfolio with roughly 8%. For the rest, not each one of the asset is more important than 3%.

Clearly, diversification is not enough in our portfolio. With our portfolio mostly contained within five companies, idiosyncratic risk remains too elevated.

6.2 Marginal Risk and Beta Contributions

Evidently, the β of the portfolio is the average of its stocks:

$$\beta_{portfolio} = \sum_{i=1}^{n} w_i \beta_i, \; i = 1, ..., n$$

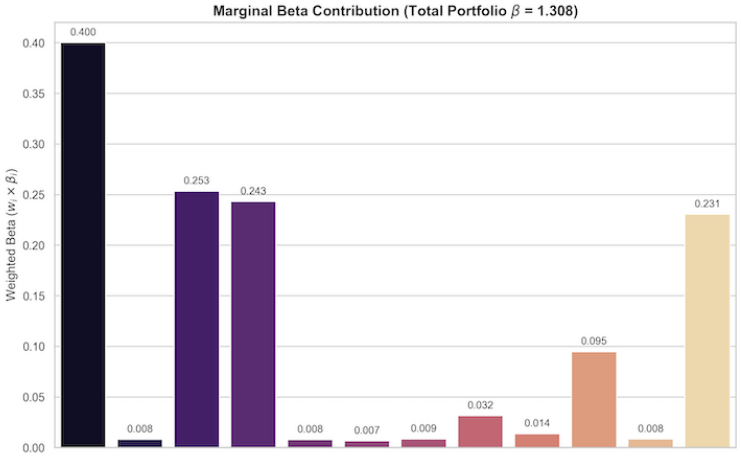

Figure 2: Beta contributions

Logically, the β contributions of Google, Apple, Tesla, Boeing, and Microsoft are the only ones that really matter. The other stocks contributing between 0.8% and 1.4%. The final β of our portfolio is about 1.31. This is considered an aggressive portfolio, even though it remains relatively close to a neutral one.

6.3 Distributional analysis

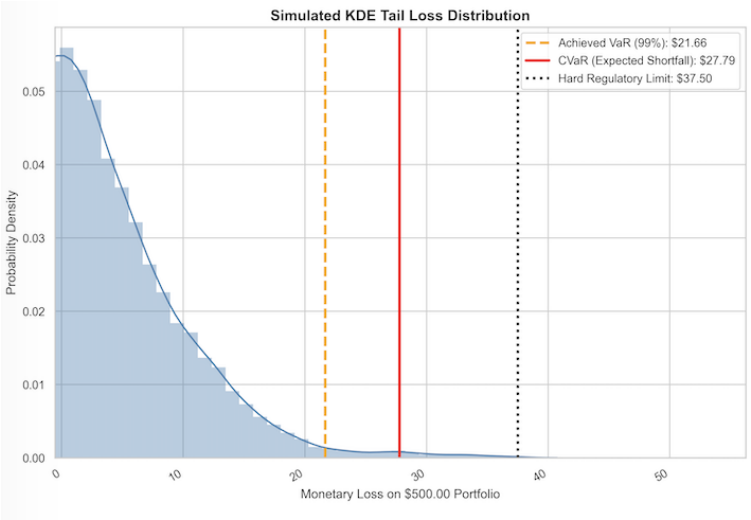

Figure 3: Simulated KDE Tail Loss Distribution

We set a MAVaR of 7.5% the value of the portfolio, being defined at $500, corresponding to $37.5. The graph here displays the empirical results of our simulations. We can see that both CVaR and VaR (at the 99% threshold) are far below this maximum loss (represented by the dotted-vertical-line on the right side of the graph).

The plateau of the risk metric significantly below the MAVaR limit implies the algorithm reached peak available β within the designated asset universe.

6.4 Historical trajectory

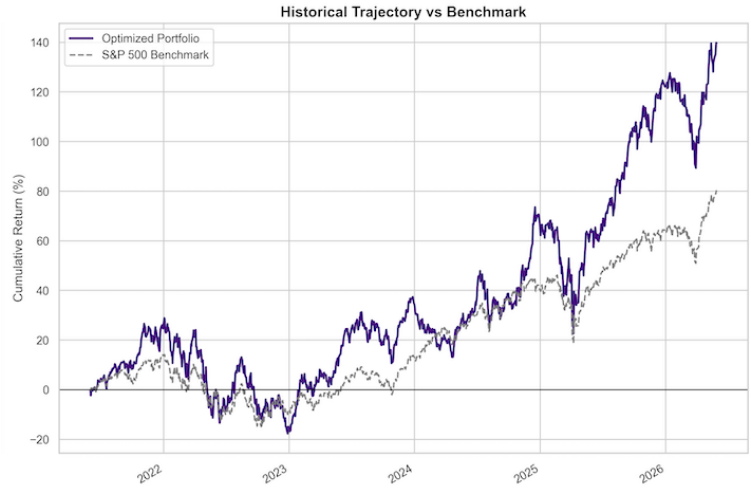

Figure 4: Portfolio returns vs. S&P500 returns

The trajectory clearly demonstrates that during market rallies (particularly from 2025 through 2026), the portfolio systematically amplifies the benchmark returns, confirming the algorithm successfully maximized market exposure.

The graph depicts the friction between β and standard volatility. During the 2022 and early 2023 market contractions, the optimized portfolio experienced a steeper drawdown than the S&P 500, dropping into negative territory. This visual proves that while our MAVaR limit contained extreme tail-risk, it did not eliminate baseline market beta drag.

It is obvious that with a greater β we get a higher risk and so higher drawdowns during market corrections. However, to evaluate more clearly the performance of our strategy, we need to backtest it.

6.5 Backtesting the strategy

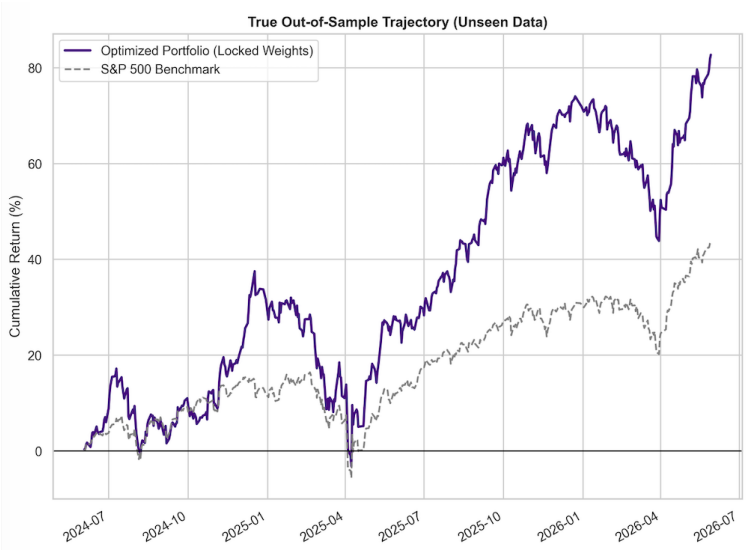

Figure 5: Portfolio returns vs. S&P500 returns — Backtested data

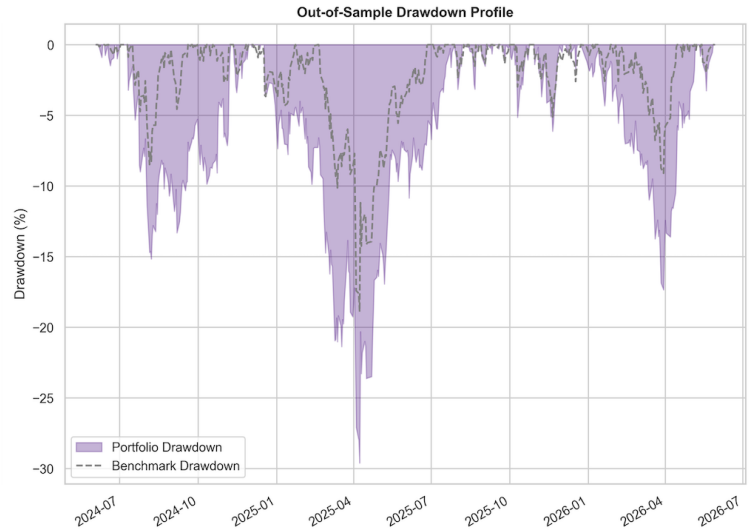

Figure 6: Drawdown comparisons — Backtested data

Our backtested data show that this strategy is efficient even though riskier. The most accurate way of estimating if this strategy has an interest is to compare the Treynor ratio of our portfolio and the one of the benchmark. First, we have:

$$T = \frac{R_p - R_f}{\beta}$$

In fact, it represents the gain of excess return for each "gain of β". To facilitate calculations, we assume that the risk-free rate is equal to zero.

Logically, the Treynor ratio of the benchmark is its return, so here it is 43.63% over the studied period. On the other side, our portfolio exhibits a Treynor ratio of 63.27%. This definitely validates the risk-taking of our strategy.

7 Conclusion

The paper successfully demonstrates that non-parametric tail-risk measures can be actively integrated into aggressive portfolio optimization. By freezing the simulation state space to eliminate stochastic noise, the SLSQP algorithm successfully converged, proving that Kernel Density Estimation (KDE) and Monte Carlo simulations can enforce strict VaR boundaries without breaking local gradient calculations.
The optimization algorithm fulfilled the core objective: constructing an asymmetric, growth-oriented portfolio ($\beta \approx 1.31$) that maximizes systematic market exposure while mathematically isolating the portfolio from catastrophic tail-risk. The achieved VaR(99%) remained well below the arbitrary $\$37.50$ maximum acceptable limit.

The true out-of-sample backtest validates the robustness of the static beta coefficients and the non-parametric risk constraints. While the portfolio naturally experienced steeper conditional drawdowns during market corrections due to its elevated β, the MAVaR constraint ensured these drawdowns remained survivable, allowing the portfolio to systematically amplify subsequent market rallies. Measured Treynor ratios deliver a 63.27% against the S&P 500 benchmark's 43.63%. The model empirically proves that the excess returns generated by the algorithm actively and sufficiently compensated for the higher systemic risk assumed.

While systematically robust, the optimal allocation revealed a heavy concentration in a narrow band of technology and aerospace equities. This lack of broad diversification leaves the model exposed to idiosyncratic, sector-specific shocks. Future research could expand upon this framework by integrating a dynamic rolling-window re-balancing mechanism, or by introducing conditional correlation matrices to better map contagion risk across specific asset classes during black swan events.

References

[1] Mazaud. Var_cvar_model: Var_cvar_simulation.py. `https://github.com/GianniMzd/VaR_CVaR_model/blob/main/VaR_CVaR_simulation.py`, 2026.